

Seguimiento IV
Electronica Digital II
Juego con obstaculos

Julian Jimenez
Valentina Padierna
Ana Zapata

6 mayo de 2026

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	4
5. Estructura del código	4
6. Explicación de funciones principales	5
7. Comunicación	6
8. Interfaz de usuario	7
9. Procedimiento de prueba	7
10. Manejo de errores y seguridad	8

1. Identificación del proyecto

- **Nombre del proyecto:** Juego Dodger en esp32 OLED+ESP32
- **Integrantes del equipo :** Ana Sofia Zapata , Valentina padierna, y Julian jimenez

2. Resumen

Nuestro videojuego Dodger desarrollado en ESP32 utilizando MicroPython y una pantalla OLED SSD1306. El jugador controla un personaje que debe esquivar obstáculos que aparecen desde el lado derecho de la pantalla y se mueven hacia la izquierda.

El juego incluye diferentes modos de juego, sonidos con buzzer, control mediante botones, sistema de puntaje y una máquina de estados para controlar las distintas pantallas del sistema.

El objetivo principal del proyecto es integrar conceptos vistos en clase como manejo de GPIO, comunicación I2C, PWM, temporización, debounce y programación estructurada en tiempo real utilizando MicroPython.

3. Descripción del hardware

El sistema está basado en una placa ESP32, la cual se encarga de controlar todas las entradas y salidas del videojuego. La ESP32 procesa las señales de los botones, genera los sonidos del buzzer, también tiene efectos visuales si pierde el jugador y controla la pantalla OLED donde se muestra el juego.

El sistema cuenta con tres botones principales que permiten controlar el movimiento del jugador y las acciones del juego.

- **Boton Up:** Conectado al GPIO.16
- **Boton Down:** Conectado al GPIO.17
- **Botón START / PAUSE:** Conectado al GPIO.14
- **Pantalla OLED SSD1306:** Conectadas a SDL GPIO.21 y SCL al GPIO.22
- **Buzzer – Salida sonora:** Al GPIO.27
- **Led-Respuesta visual :** GPIO.2

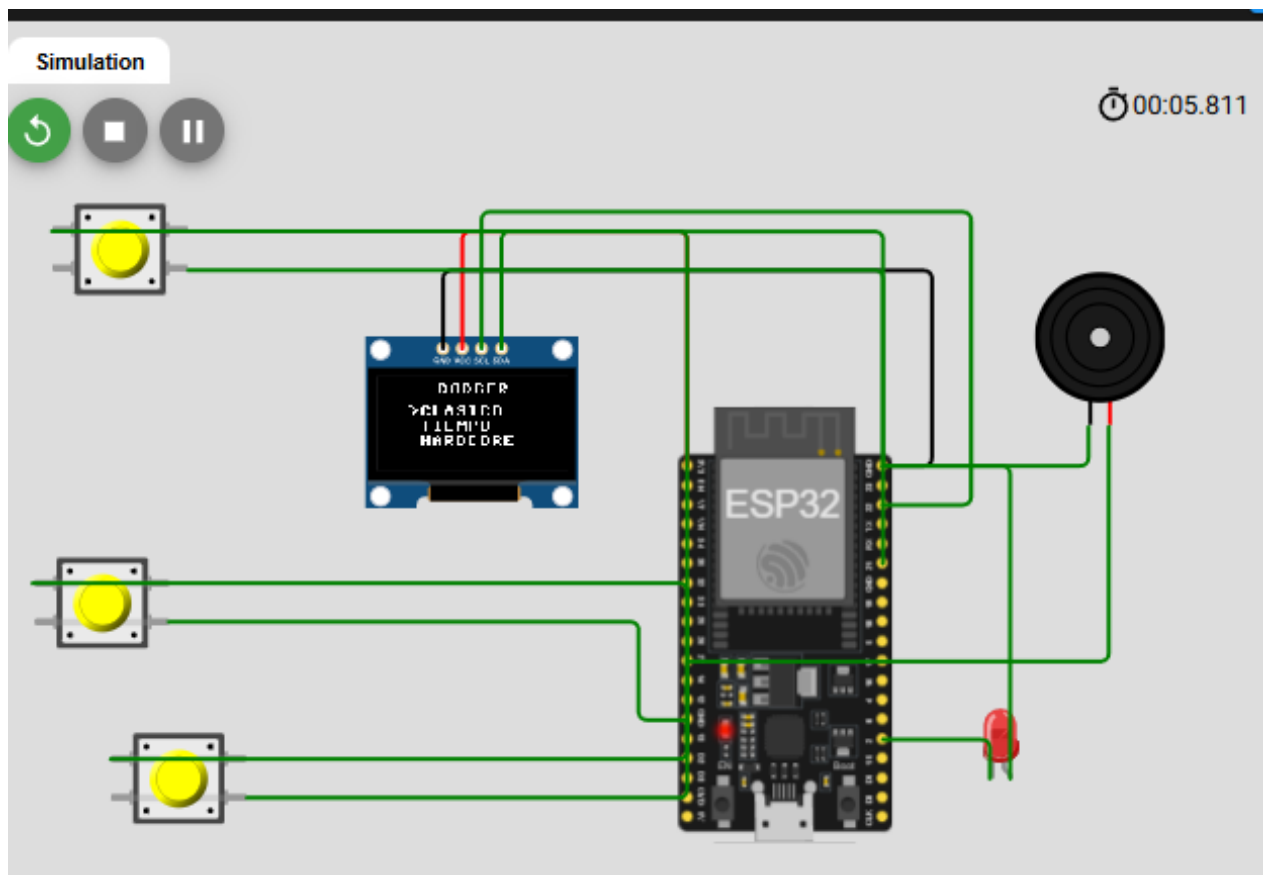


Figura 1: Diagrama de conexiones

4. Descripción del software

- Lenguaje utilizado: MicroPython
- IDE utilizado: Thonny

Librerías principales

- machine → control de GPIO, I2C y PWM
- ssd1306 → manejo de pantalla OLED
- framebuf → renderizado gráfico
- time → temporización
- random → generación aleatoria
- Funcionamiento general del software

El programa inicia configurando todos los componentes del hardware. Después, muestra un menú principal donde el usuario puede seleccionar el modo de juego.

Durante la partida, el sistema genera obstáculos aleatorios, detecta colisiones y actualiza el puntaje en tiempo real.

El programa utiliza temporización basada en `time.ticksms()` para evitar bloqueos y permitir que varias tareas ocurran al mismo tiempo.

Librería SSD1306

La pantalla OLED funciona utilizando una librería SSD1306 implementada en MicroPython.

La librería crea un framebuffer donde se almacenan temporalmente los gráficos antes de ser enviados a la pantalla mediante I2C.

La clase `SSD1306I2C` permite manejar la comunicación directa con la pantalla utilizando:

5. Estructura del código

El proyecto está organizado en un único archivo principal llamado `main.py`. El código se divide en diferentes secciones:

- Configuración de hardware
- Configuración de debounce
- Variables globales
- Sonido
- Obstáculos
- Disparos
- Colisiones
- Renderizado gráfico
- Máquina de estados
- Loop principal

La programación se realizó utilizando funciones para facilitar la organización y comprensión del sistema

6. Explicación de funciones principales

Función `ispressed()` Esta función se desarrolló para detectar las pulsaciones de los botones y evitar errores causados por el rebote mecánico de los mismos.

El sistema utiliza un antirebote (debounce) por software mediante la función `ticksms()`, permitiendo marcar como pulsaciones efectivas únicamente las pulsaciones realizadas después de cierto intervalo de tiempo(150ms).

Adicionalmente, la función implementa un modo hold, el cual permite repetir acciones (subir-bajar) mientras el botón permanece presionado. Esto fue utilizado para lograr un movimiento continuo del jugador.

Función `buzzer_start()`

La función de `buzzerstart()`, fue implementada para el manejo de encendido y apagado del buzzer (retroalimentación sonora). Y lo hace a través de parámetros como la frecuencia del sonido y de la duración en milisegundos de este, permitiendo así generar diferentes tonos dependiendo de los eventos del juego dado que se les varía los parámetros con los que funciona la función. Eventos dentro del juego * navegación en el menú, * colisiones, * disparos, * obtención de puntos.

Funcion `update_difficulty()`

Esta función se implementó para poder realizar el aumento progresivo de la dificultad del juego con respecto al tiempo que transcurra en este mismo y también con el modo seleccionado. Además, también se implemento para poder hacer el juego mas dinámico y no tan

estático, entonces gradualmente se aumenta la intensidad de este.

En esta función se modifican variables del juego como, la velocidad de los obstáculos y cada cuanto aparecen estos.

Función `spawn_obstacle()`

Esta función en nuestro juego permite la creación de obstáculos de manera aleatoria.

Se crea la posición vertical en la que deben estar los obstáculos utilizando la librería `random`; y cada obstáculo se almacena como un diccionario que contiene: posición horizontal, posición vertical, ancho y altura. Y es con esto que se pueden generar múltiples obstáculos y de una forma no repetitiva, lo que hace que la secuencia del juego sea mucho mas entretenida.

Función `check_player_collision()`

Es con esta función que podemos verificar si hubo o no colisión entre el jugador y los obstáculos, y por consiguiente, se verifica si el jugador gana o no puntos y el finalizar de la partida. Esta detección se hace comparando coordenadas en las que esta el jugador con respecto a las dimensiones de cada obstáculo generado. Cuando se detecta una colisión se genera retroalimentación visual y auditiva como: el activo del buzzer y el encendido del led, además de que el sistema cambia a `GAMEOVER`.

Función `game()`

Dentro de esta función se realizan distintas tareas que permiten establecer la lógica principal del juego, como:

- Lectura de botones
- Movimiento del jugador
- Creación de obstáculos
- Detección de colisiones
- Aumento de dificultad
- Actualización de pantalla

Es esta función de `game`, que como su nombre lo indica, es la que nos permite llevar a cabo el juego, es como ese ciclo que estamos ejecutando múltiples veces cada vez que pulsamos el `start`.

7. Comunicación

La comunicación entre la ESP32 y la pantalla OLED se realiza mediante protocolo I2C.

La librería SSD1306 utiliza el método: `i2c.writeto()` para enviar comandos y datos gráficos al display OLED.

Comunicación serial

La ESP32 también se comunica con el computador mediante UART vía USB utilizando Thonny.

Esto permite:

Subir el código Ejecutar pruebas Ver errores Realizar depuración del sistema

8. Interfaz de usuario

La interfaz combina elementos físicos y gráficos.

Interfaz física

- El usuario interactúa mediante tres botones físicos conectados a la ESP32.

Interfaz gráfica

- La pantalla OLED muestra:
 - Menú principal
 - Modos de juego
 - Puntaje
 - Tiempo
 - Obstáculos
 - Pantalla de pausa
 - Pantalla de game over
 - Retroalimentación sonora

El buzzer genera sonidos diferentes dependiendo de las acciones del jugador y los eventos del juego.

9. Procedimiento de prueba

Prueba de botones

Se verificó el correcto funcionamiento de los botones y del debounce por software.

Prueba de pantalla OLED

Se comprobó el renderizado correcto de:

- Texto
- Sprites
- Obstáculos

- HUD

Prueba de buzzer

Se realizaron pruebas con diferentes frecuencias y duraciones de sonido.

Prueba de colisiones

Se verificó que el sistema detectara correctamente las colisiones entre el jugador y los obstáculos.

Prueba de modos de juego

Se probaron individualmente los modos:

- Clásico
- Contra tiempo
- Hardcore

verificando dificultad, velocidad y funcionamiento general.

10. Manejo de errores y seguridad

También intentamos evitar errores mientras hacíamos el juego.

A los botones les pusimos antirrebote para que una sola pulsación no se detectara muchas veces. También limitamos el movimiento del jugador para que no se saliera de la pantalla.

El juego corre todo el tiempo dentro del while True, entonces nunca se queda quieto ni congelado.

Además, cuando desaparecen obstáculos o disparos, se eliminan correctamente para que no den errores. Y cuando se reinicia la partida, también se reinician variables como el puntaje, el tiempo y los obstáculos.

En general, buscamos que el juego funcionara estable mientras se juega.